Estimation: Bid Intelligence Platform

ReMarkets — Bid Intelligence Platform

Option B — Blazor Web App + .NET 10

Scope Summary

Internal web application for REMarkets to replace spreadsheet/email-based bidding workflows. Covers bid management, inventory handling, allocation approvals, role-based access, act-on-behalf, audit trails, and order-ready exports. Authentication via Microsoft Entra. All data stored in clean, normalized SQL for future analytics consumption.

Architecture Summary

- Frontend + Backend: Blazor Web App (.NET 10) — unified C# codebase
- Render Modes: Static SSR for simple pages, Interactive Server for bid tables and allocation views
- UI Components: MudBlazor (free) with DataGrid for in-cell editing
- Database: Azure SQL Database with Entity Framework Core 10 (Code-First)
- Auth: Microsoft Entra ID via Microsoft.Identity.Web (built-in Blazor auth)
- Hosting: Azure App Service (3 environments: Dev, Test, Prod)
- Deploy: Single project, single deploy

Data Grid Library Options

The in-cell editing grid is the most critical UI component — it's where sales reps enter bids, quantities, and prices directly in a table (like Excel in the browser).

Library	License	Cost	In-Cell Editing	Notes
MudBlazor DataGrid (Recommended)	MIT (free, open source)	$0	Good	Active community, evaluate Sprint 0
Radzen DataGrid	MIT (free, open source)	$0	Good	Alternative to MudBlazor
AG Grid (Blazor wrapper)	Community: MIT / Enterprise: Commercial	~$1,100/dev	Best-in-class	Same AG Grid as React
Telerik UI for Blazor	Commercial (paid license)	~$1,000/dev/yr	Excellent	Full UI suite + professional support
Syncfusion Blazor	Community: free <$1M rev	$0 or ~$995/dev/yr	Very good	Free tier may apply
Manual (jQuery/JS)	N/A	$0	Painful	NOT recommended — conflicts with Blazor DOM

Recommendation: Start with MudBlazor (free, MIT). If Sprint 0 shows it's insufficient, upgrade to Telerik or AG Grid (~$2–3K for 2–3 devs).

MVP Definition

Core question: Can REMarkets run 100% of internal bid-to-allocation workflows through the platform instead of spreadsheets and email?

MVP goal: A working web application where sales reps can create offers, enter bids, upload inventory, and route allocations through an approval workflow — with full audit logging and role-based access.

MVP Scope (Must Have)

Without these, the project fails to replace the current spreadsheet/email process.

#	Module	Task	Complexity	Base Hrs	Factor	Adjusted	Notes
Project Setup							
1	Setup	Blazor Web App project scaffolding (Clean Architecture)	Low	12	×1.2	**14**	Single project, C# only
2	Setup	EF Core setup, initial migrations, seed data	Low	12	×1.2	**14**	
3	Setup	CI/CD pipeline (build + test + deploy)	Low	10	×1.2	**12**	Single project = simpler pipeline
Authentication & Authorization — Option A: Azure AD Groups (Confirmed)							
4	Auth	Entra integration — Microsoft.Identity.Web (built-in Blazor auth)	Medium	12	×1.5	**18**	Simpler than MSAL.js
5	Auth	Azure AD Group setup + group claims mapping (5 groups)	Medium	10	×1.5	**15**	Configure groups in Azure AD
6	Auth	Dynamic role-based authorization + app-layer enforcement	Medium	14	×1.5	**21**	Policies reading AD group claims
7	Auth	AuthorizeView components (dynamic role-based UI rendering)	Low	12	×1.2	**14**	Show/hide based on group membership
Customer Management							
8	Customers	Customer CRUD (API + Blazor pages)	Low	14	×1.2	**17**	Standard CRUD
9	Customers	Customer assignment to Sales Reps/Managers	Low	8	×1.2	**10**	
10	Customers	Customer deactivation (soft delete) + audit logging	Low	8	×1.2	**10**	
Inventory Management							
11	Inventory	Bulk CSV/Excel upload (parsing + validation + error handling)	High	24	×2.0	**48**	File parsing edge cases
12	Inventory	Inventory identity — composite uniqueness + master vs source description	High	20	×2.0	**40**	Client-confirmed composite key
12b	Inventory	Master description edit UI (search + inline edit + role restriction)	Low	6	×1.2	**8**	SC-002 A15: dedicated UI, audited
13	Inventory	Inventory adjustment records (each upload = distinct transaction)	Medium	14	×1.5	**21**	Full audit trail per upload row
14	Inventory	Inventory list view with state tracking (Available/Committed/Released)	Medium	14	×1.5	**21**	
15	Inventory	Inventory grouping into offers	Medium	12	×1.5	**18**	
16	Inventory	State transitions (automatic on approval + manual override)	Medium	16	×1.5	**24**	Business rules complexity
Bid & Offer Management							
17	Bids	Create/Edit offers (Blazor forms)	Medium	14	×1.5	**21**	

#	Module	Task	Complexity	Base Hrs	Factor	Adjusted	Notes
18	Bids	Offer list view with filtering and sorting	Medium	10	×1.5	**15**	
19	Bids	MudBlazor DataGrid — in-cell editing for line-item bids	High	28	×2.0	**56**	Core feature, DataGrid config
20	Bids	Bid uniqueness enforcement + revision tracking	Medium	10	×1.5	**15**	One active bid per customer/line/offer
21	Bids	Bid types support (bid/BIN/target pricing)	Medium	12	×1.5	**18**	
22	Bids	Winning/losing bid capture and status tracking	Medium	12	×1.5	**18**	
23	Bids	Bid history (view previous revisions per line item)	Medium	14	×1.5	**21**	
Allocation & Approval Workflow							
24	Allocation	Aggregated allocation view (all bids per offer)	Medium	14	×1.5	**21**	
25	Allocation	Admin override of winning customer	Medium	12	×1.5	**18**	
26	Allocation	Approval workflow (Pending → Approved/Rejected) — all manager-approved Phase 1	High	24	×2.0	**48**	Admin has full authority incl. override
27	Allocation	Visual pricing indicators (“below floor” warnings)	Medium	8	×1.5	**12**	Visual only Phase 1, no auto-reject
28	Allocation	Automatic inventory commitment on approval	Medium	12	×1.5	**18**	
Allocation Reversal (Client-confirmed)							
29	Allocation	Allocation reversal (return inv. to Available, mark Reversed, lock export)	High	20	×2.0	**40**	State machine + inventory rollback
30	Allocation	Reversal reason code (required) + full audit trail	Medium	8	×1.5	**12**	Reason code dropdown + audit
Allocation Tie-Breaker (Client-confirmed)							
31	Allocation	Tie detection logic + resolution UI	Medium	8	×1.5	**12**	Admin picks winner from tied bids
Act-on-Behalf (Client-requested — Kim: “very important”)							
32	Act-on-Behalf	Act-on-behalf capability — full attribution + audit	High	24	×2.0	**48**	Custom AuthenticationStateProvider
33	Act-on-Behalf	Act-on-behalf UI (customer selector + visual indicator)	Medium	10	×1.5	**15**	Clear visual cue
Split Allocations (Client-requested)							
34	Allocation	Partial/split allocations across customers	High	24	×2.0	**48**	Manual decision by manager
Audit & Compliance							
35	Audit	Audit logging middleware (insert-only AuditLog table)	Medium	16	×1.5	**24**	
36	Audit	Audit trail UI (view history per entity)	Medium	14	×1.5	**21**	
Order-Ready Exports							
37	Exports	CSV export from approved allocations	Low	8	×1.2	**10**	
Permissions UI							

#	Module	Task	Complexity	Base Hrs	Factor	Adjusted	Notes
38	Permissions	Role-based UI visibility (AuthorizeView across pages)	Medium	14	×1.5	**21**	
Testing & Bug Buffer							
39	QA	Unit tests (services + critical business logic)	Medium	20	×1.5	**30**	
40	QA	Integration tests (API/page endpoints)	Medium	14	×1.5	**21**	
41	QA	Bug fixing buffer (10% of MVP hours)	—	—	—	**90**	Based on ~900 hrs pre-buffer
					MVP Total (Auth Option A)	**~998 hrs**	

Post-MVP Scope (Should Have)

Features that add value after MVP is stable.

#	Module	Task	Complexity	Base Hrs	Factor	Adjusted	Source
42	Allocation	Escalation paths (auto-escalate if not approved in X time)	Medium	16	×1.5	**24**	Suggested enhancement
43	Exports	PDF export (order-ready documents via QuestPDF)	Medium	14	×1.5	**21**	Client mentioned; CSV is in MVP
44	Audit	Advanced audit views (filter by user, date, entity type)	Medium	10	×1.5	**15**	Suggested enhancement
45	Admin	System configuration UI (thresholds, settings)	Medium	14	×1.5	**21**	Suggested enhancement
46	QA	UAT support + bug fixes from user feedback	Medium	24	×1.5	**36**	Standard
					Post-MVP Total	**~117 hrs**	

Backlog (Could Have)

#	Task	Source
47	Dashboard with key metrics (active offers, pending approvals, recent bids)	Suggested enhancement
48	Bulk bid operations (approve/reject multiple at once)	Suggested enhancement
49	Advanced search and filtering across all entities	Suggested enhancement
50	Email notifications on approval status changes	Suggested enhancement
51	Data export API for future external consumption	Suggested enhancement
52	User activity log (who logged in when)	Suggested enhancement

Out of Scope (Won’t Have)

- Customer-facing portal (future phase)
- In-app analytics dashboards (future consideration)
- Mobile application
- Integration with external ERP or order management systems
- Email/notifications

Learning Curve Budget

Technology	Who	Hours	Justification
Blazor component model + render modes	Dev 1	12	Components, lifecycle, SSR vs Interactive
Blazor component model + render modes	Dev 2	12	Same
MudBlazor (DataGrid + UI components)	Dev 1	8	Grid API, cell editing, forms
MudBlazor (DataGrid + UI components)	Dev 2	6	Support knowledge
Microsoft.Identity.Web (Blazor auth)	Dev 1	6	Built-in auth patterns
Azure App Service + SQL (if new)	Both	8	Deployment, config
CI/CD pipeline setup	TBD	4	Single project = simpler

Learning Curve Total: ~56 hrs

Risk Assessment

Risk	Probability	Impact	Hours at Risk	Mitigation
MudBlazor DataGrid insufficient for in-cell editing	Medium	High	+16-24 hrs	Evaluate Sprint 0; Telerik/Syncfusion fallback
SignalR connection reliability	Low	Medium	+8-16 hrs	Built-in reconnection; test under real network
Blazor community smaller	Medium	Medium	+8-16 hrs	MudBlazor Discord, Microsoft docs
Partial/split allocation rules unclear	Low	High	+24-40 hrs	SC-003 resolved: auto-release, no reservation, flat records
Allocation reversal edge cases	Medium	High	+16-24 hrs	Define reversal rules with POs; test with real scenarios
Inventory composite uniqueness conflicts	Medium	Medium	+8-16 hrs	Define exact rules with client; validate Sprint 0
CSV upload edge cases	Medium	Medium	+8-16 hrs	Define validation rules; limit file sizes
Azure infra not available on time	Medium	High	Blocks 1-2 wk	Confirm access before Sprint 1
Scope creep from 3 Product Owners	Medium	High	+40-80 hrs	Reference charter; change request process
Open discovery questions (21 unresolved)	High	High	+40-150 hrs	Prioritize before Sprint 1; see discovery-questions.md

Summary

Concept	Hours
MVP (Must Have) — Auth Option A: Azure AD Groups	~998 hrs
MVP (Must Have) — Auth Option B: Internal DB Roles	~1,022 hrs
Post-MVP (Should Have)	~117 hrs
Learning Curve	~56 hrs
Full Scope (MVP + Post-MVP + Learning) — Auth Option A	~1,171 hrs
Full Scope (MVP + Post-MVP + Learning) — Auth Option B	~1,195 hrs

Updated Feb–Mar 2026 after client Q&A. SC-001: +146 hrs. SC-002: +8 hrs. SC-003: 0 hrs (design decisions only, no new scope).

Timeline Scenarios — Auth Option A (2 developers, 80 hrs/week)

Scenario	Scope	Hours	Calendar Estimate
MVP only	Must Have + Learning	~1054 hrs	~13.2 weeks (~3.3 months)
MVP + Post-MVP	Full Phase 1	~1,171 hrs	~14.6 weeks (~3.7 months)

With 3 developers (120 hrs/week)

Scenario	Scope	Hours	Calendar Estimate
MVP only	Must Have + Learning	~1054 hrs	~8.8 weeks (~2.2 months)
MVP + Post-MVP	Full Phase 1	~1,171 hrs	~9.8 weeks (~2.4 months)

Auth Option B adds ~24 hrs to all scenarios. Timelines assume devs are productive from week 1 after learning curve. Actual ramp-up may add 1–2 weeks.

Head-to-Head: React vs Blazor

Metric	React + .NET 10	Blazor + .NET 10	Difference
MVP Hours (Auth Option A)	~1,100 hrs*	~998 hrs	Blazor saves ~102 hrs
Post-MVP Hours	~126 hrs	~117 hrs	Blazor saves ~9 hrs
Learning Curve	~100 hrs	~56 hrs	Blazor saves ~44 hrs
Total Hours (Auth Option A)	~1,326 hrs*	~1,171 hrs	Blazor saves ~155 hrs
MVP Timeline (2 devs)	~15 weeks*	~13.2 weeks	~1.8 weeks faster
Full Scope Timeline (2 devs)	~16.5 weeks*	~14.6 weeks	~1.9 weeks faster
In-cell editing quality	Best (AG Grid)	Good (MudBlazor)	React has better grid
Future customer portal	Excellent	Good	React better

Metric	React + .NET 10	Blazor + .NET 10	Difference
			long-term
Hiring future devs	Excellent	Limited	React market larger
CI/CD complexity	Medium (2 deploys)	Low (1 deploy)	Blazor simpler

** React estimation not yet updated with client Q&A scope changes. Numbers are projected proportional estimates.*

Bottom line: Blazor saves ~160 hours and ~2 weeks on timeline. React costs more upfront but offers better in-cell editing, easier hiring, and a stronger path to a future customer portal.

Team Sign-off

Name	Role	Approved
TBD	Simpat Lead	
TBD	Dev 1	
TBD	Dev 2	